

Machine Learning for Natural Language Processing

Representing positions in Transformers



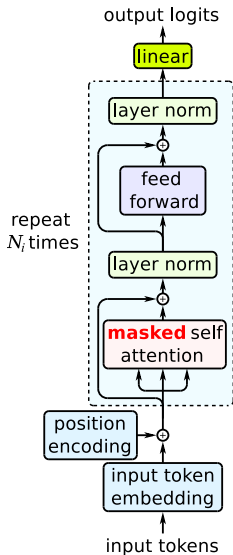
UNIVERSITY OF
GOTHENBURG

CHALMERS

Richard Johansson

`richajo@chalmers.se`

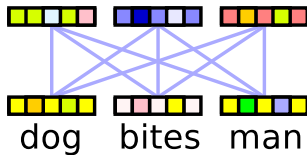
The Transformer architecture



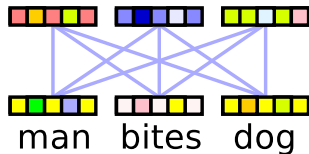
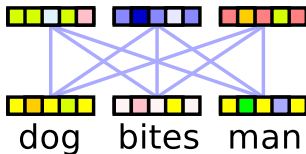
The representation of **positions** varies between Transformer implementations

One of the main differences between earlier and modern Transformers

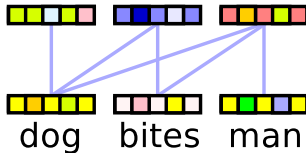
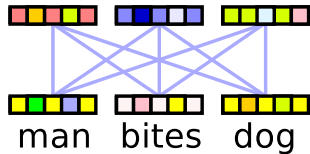
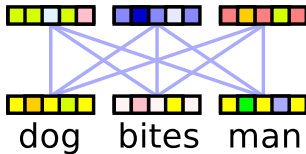
Symmetries in the Transformer



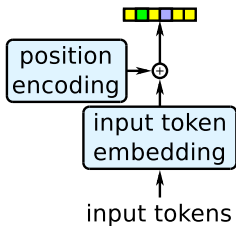
Symmetries in the Transformer



Symmetries in the Transformer



Positions in early Transformers



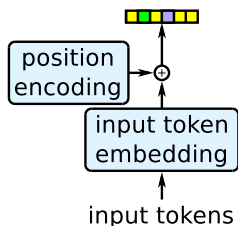
Early Transformers added **absolute positional encodings** (or positional embeddings) at the bottom layer

Original Transformer: Sinusoidal embeddings

The **Attention is all you need** paper (Vaswani et al., 2017) used a deterministic approach based on trigonometric functions

$$\text{PE}_{2i}(p) = \sin\left(p \cdot 10000^{-\frac{2i}{d}}\right) \quad \text{PE}_{2i+1}(p) = \cos\left(p \cdot 10000^{-\frac{2i}{d}}\right)$$

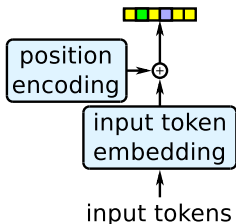
Learned position embeddings



GPT-1/2/3 (Radford et al., 2018) and BERT (Devlin et al., 2019) used **learned** position embeddings

One embedding for every unique position (up to max length)

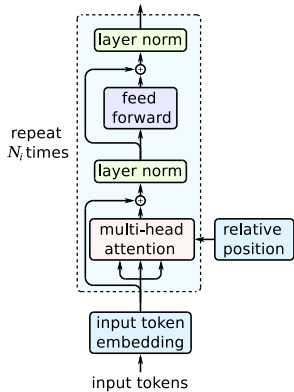
Fixed or learned: pros and cons



Fixed sinusoidal: nothing to learn, model may (theoretically) learn to **extrapolate** to longer sequences

Learned: easy to code, works well in practice, but does not extrapolate

Positions in modern Transformers



No token-level position embeddings:
instead, the **attention** module takes
position into account

Position representations are **relative**

Rotary Position Embedding (RoPE, Su et al., 2021)

RoFORMER: ENHANCED TRANSFORMER WITH ROTARY POSITION EMBEDDING

Jianlin Su
Zhuiyi Technology Co., Ltd.
Shenzhen
bojonesu@wezhuiyi.com

Yu Lu
Zhuiyi Technology Co., Ltd.
Shenzhen
julianlu@wezhuiyi.com

Shengfeng Pan
Zhuiyi Technology Co., Ltd.
Shenzhen
nickpan@wezhuiyi.com

Ahmed Murtadha
Zhuiyi Technology Co., Ltd.
Shenzhen
mengjiayi@wezhuiyi.com

Bo Wen
Zhuiyi Technology Co., Ltd.
Shenzhen
brucewen@wezhuiyi.com

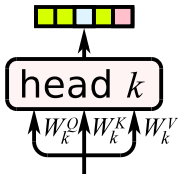
Yunfeng Liu
Zhuiyi Technology Co., Ltd.
Shenzhen
glenliu@wezhuiyi.com

November 9, 2023

ABSTRACT

Position encoding recently has shown effective in the transformer architecture. It enables valuable supervision for dependency modeling between elements at different positions of the sequence. In this paper, we first investigate various methods to integrate positional information into the learning process of transformer-based language models. Then, we propose a novel method named Rotary Position Embedding (RoPE) to effectively leverage the positional information. Specifically, the proposed RoPE encodes the absolute position with a rotation matrix and meanwhile incorporates the explicit relative position dependency in self-attention formulation. Notably, RoPE enables valuable properties, including the flexibility of sequence length, decaying inter-token dependency with increasing relative distances, and the capability of equipping the linear self-attention with relative position encoding. Finally, we evaluate the enhanced transformer with rotary position embedding, also called RoFormer, on various long text classification benchmark datasets. Our experiments show that it consistently overcomes its alternatives. Furthermore, we provide a theoretical analysis to explain some experimental results. RoFormer is already integrated into Huggingface: https://huggingface.co/docs/transformers/model_doc/roformer.

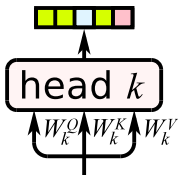
RoPE key ideas



Recall: Attention heads compute key, query, and value representations

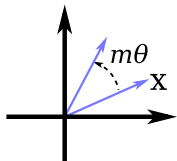
RoPE operates on the **key and query vectors** in attention

RoPE key ideas



Recall: Attention heads compute key, query, and value representations

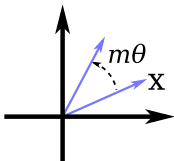
RoPE operates on the **key and query vectors** in attention



It applies **rotation matrices** where the rotation angle depends on the position

In the 2-dimensional case:

$$\text{RoPE}(x, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$



RoPE: more mathematical details

In the d -dimensional case:

$$\begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix}$$

Because of the rotations, attention between positions m and n depends on $m - n$: the **relative** position

$$\mathbf{q}_m^\top \mathbf{k}_n = (\mathbf{R}_{\Theta, m}^d \mathbf{W}_q \mathbf{x}_m)^\top (\mathbf{R}_{\Theta, n}^d \mathbf{W}_k \mathbf{x}_n) = \mathbf{x}_m^\top \mathbf{W}_q^\top \mathbf{R}_{\Theta, m-n}^d \mathbf{W}_k \mathbf{x}_n$$

The ALiBi method (Press et al., 2022)

TRAIN SHORT, TEST LONG: ATTENTION WITH LINEAR BIASES ENABLES INPUT LENGTH EXTRAPOLATION

Ofir Press^{1,2} Noah A. Smith^{1,3} Mike Lewis²

¹Paul G. Allen School of Computer Science & Engineering, University of Washington

²Facebook AI Research

³Allen Institute for AI

ofirp@cs.washington.edu

ABSTRACT

Since the introduction of the transformer model by Vaswani et al. (2017), a fundamental question has yet to be answered: how does a model achieve extrapolation at inference time for sequences that are longer than it saw during training? We first show that extrapolation can be enabled by simply changing the position representation method, though we find that current methods do not allow for *efficient* extrapolation. We therefore introduce a simpler and more efficient position method, Attention with Linear Biases (ALiBi). ALiBi does not add positional embeddings to word embeddings; instead, it biases query-key attention scores with a penalty that is proportional to their distance. We show that this method trains a 1.3 billion parameter model on input sequences of length 1024 that extrapolates to input sequences of length 2048, achieving the same perplexity as a sinusoidal position embedding model trained on inputs of length 2048 but training 11% faster and using 11% less memory. ALiBi's inductive bias towards recency also leads it to outperform multiple strong position methods on the WikiText-103 benchmark.¹

For decoder models, do we need positions?

Transformer Language Models without Positional Encodings Still Learn Positional Information

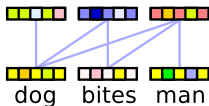
Adi Haviv[†] Ori Ram[†] Ofir Press^ω Peter Izsak[‡] Omer Levy^{†μ}

[†] Tel Aviv University

^ω University of Washington

[‡] Intel Labs

^μ Meta AI



Abstract

Causal transformer language models (LMs), such as GPT-3, typically require some form of positional encoding, such as positional embeddings. However, we show that LMs without any explicit positional encoding are still competitive with standard models, and that this phenomenon is robust across different datasets, model sizes, and sequence lengths. Probing experiments reveal that such models acquire an implicit notion of absolute positions throughout the network, effectively compensating for the missing information. We conjecture that causal attention enables the model to infer the number of predecessors that each token can attend to, thereby approximating its absolute position. Our findings indicate that causal LMs might derive positional awareness not only from the explicit positioning mechanism, but also from the effects of the causal mask.

1 Introduction

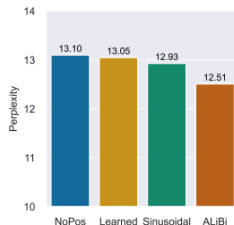


Figure 1: Transformer language models trained without explicitly encoding positional information (*NoPos*) approach the performance of models trained with various positional encoding methods. All models have 1.3B parameters, and are trained on an excerpt of the Pile.

Takeaways and further notes

Transformers differ in their **representation of positions**

As of 2025, **RoPE** is the dominant method: used by Llama, Qwen, Olmo, gpt-oss, DeepSeek, ...

Later, we will discuss how to process **very long sequences**

Side note: all methods described here have **2D counterparts**

References

- J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. 2019. [BERT: Pre-training of deep bidirectional transformers for language understanding](#). In *NAACL*.
- A. Haviv, O. Ram, O. Press, P. Izsak, and O. Levy. 2022. [Transformer language models without positional encodings still learn positional information](#). In *EMNLP Findings*.
- O. Press, N. A. Smith, and M. Lewis. 2022. [Train short, test long: Attention with linear biases enables input length extrapolation](#). In *ICLR*.
- A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. 2018. [Improving language understanding by generative pre-training](#). *OpenAI blog* .
- J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu. 2021. [Roformer: Enhanced transformer with rotary position embedding](#). *arXiv preprint arXiv:2104.09864* .
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. 2017. [Attention is all you need](#). In *NIPS 30*.